

Fleet Inventory — Documentación Técnica

Versión analizada: fleet_inventory_2.html

Fecha: 2026-06-04

Propósito: Herramienta web offline para cruzar datos de inventario de flota vehicular, generar reportes XLSX y visualizar/filtrar los datos resultantes.

1. Arquitectura General

El aplicativo es un **SPA (Single Page Application) monolítico** contenido en un único archivo HTML. Toda la lógica, estilos y markup conviven en el mismo fichero sin dependencias locales externas — solo una librería CDN:

Dependencia	Versión	Fuente	Uso
xlsx.full.min.js	0.18.5	cdnjs	Leer/escribir archivos Excel (XLSX)
JetBrains Mono	—	Google Fonts	Fuente monoespaciada (datos, labels)
Syne	—	Google Fonts	Fuente UI (títulos, tabs, botones)

Modelo de capas

UI / HTML (estructura visual) - NAV (tabs) - PAGE: Exportar - PAGE: Visor
CSS (design system custom con CSS variables)
JavaScript (lógica de negocio inline) - Tab Navigation - Export Module (CSV parser + procesador) - Visor Module (filtros + tabla + paginado)

2. Design System — Variables CSS

Todas las primitivas visuales están definidas en `:root` mediante custom properties:

Paleta de colores

Variable	Valor	Uso
--bg	#08090d	Fondo global
--s1	#0f1118	Superficie nivel 1 (cards)
--s2	#161a24	Superficie nivel 2 (inputs, sidebar)
--s3	#1e2333	Superficie nivel 3 (thead, select)

--b1	#252d42	Borde primario
--b2	#303a56	Borde secundario
--cyan	#00d4ff	Acento principal (activo, highlight)
--cyan2	#0098cc	Acento secundario (gradiente botón)
--green	#00e676	Estado OK / success
--amber	#ffc107	Advertencia / GEN1
--red	#ff4560	Error
--purple	#a78bfa	Acento decorativo / GEN2
--t1	#f0f2ff	Texto primario
--t2	#8b96b8	Texto secundario
--t3	#4a5270	Texto terciario (labels)
--t4	#272f48	Texto cuaternario (placeholders, row num)

Tipografía y forma

Variable	Valor
--mono	'JetBrains Mono', monospace
--ui	'Syne', sans-serif
--r	8px (border-radius base)

Efecto de fondo

`body::before` crea un degradado radial ambiental con dos focos de luz:

- Cyan al 7% de opacidad, esquina superior izquierda
- Purple al 5% de opacidad, esquina inferior derecha

3. Componentes UI (clases CSS)

3.1 Navegación .nav

- `position: sticky; top: 0` — pegado al scroll
- `backdrop-filter: blur(14px)` — efecto glassmorphism
- `.tab` — botón de pestaña: 16px padding vertical, transición `border-bottom` cyan al activarse
- `.tab.active` — borde inferior cyan + color cyan
- `.badge` — chip contador de registros (family mono)

3.2 Cards de archivo .fc

Tarjetas para cada fuente CSV. Estados:

Clase CSS	Estado	Indicador visual
.fc (sin modificador)	Sin cargar	Borde neutro, sin barra top
.fc.ok	Cargado OK	Borde verde tenue, barra top verde
.fc.err	Error	Borde rojo tenue, barra top roja

Estructura interna: `.fc-top` (icono + nombre + rol) → `.fc-st` (status text) → `.fc-btn` (label con input file oculto).

3.3 Stats `.stat`

Cuatro métricas en flex-wrap. Colores semánticos via clases:

- `.stat-v.c` → cyan (total)
- `.stat-v.g` → green (con serial)
- `.stat-v.a` → amber (con SIM Guardian)
- `.stat-v.p` → purple (con IMEI GPS)

3.4 Progress Bar `.prog-wrap`

Oculto por defecto (`display:none`). Al procesar:

- `.prog-fill` anima `width` de 0% a 100% con transición 0.25s
- Gradiente cyan → purple

3.5 Log `.log`

Panel de auditoría, oculto por defecto. Aparece al cargar el primer archivo. Cada línea `.ll` tiene timestamp `.ts` y mensaje `.m`. Niveles:

- `.ll.ok` → verde
- `.ll.warn` → amber
- `.ll.err` → rojo
- `.ll.info` → gris claro

3.6 Toast `.toast`

Notificación flotante bottom-right. `opacity:0 + translateY(14px)` por defecto. Al añadir clase `.show` hace fade-in + slide-up. Se destruye a los 3.5s. Variantes `.ok` (verde) y `.err` (rojo).

3.7 Botones

Clase	Apariencia	Uso
.btn-primary	Gradiente cyan, texto negro	Acción principal (Procesar)
.btn-green	Gradiente green, texto negro	Exportar / Ver Visor
.btn-ghost	Transparente + borde, texto gris	Acciones secundarias (Limpiar)

3.8 Tabla `.table-panel`

Layout:

```
.table-panel
├─ .tbl-header (meta info + select de page size)
├─ .tbl-scroll (overflow-x:auto + max-height calc)
│   └─ table > thead + tbody
└─ .pager (paginación)
```

Thead sticky (`position:sticky; top:0; z-index:2`). Columnas sortables con indicador `.si` :

- Sin orden: `↕` (opacity 25%)
- Ascendente: `↑` cyan
- Descendente: `↓` cyan

Celda especial `td.rn` : número de fila, alineado derecha, color `--t4` .

4. Módulo Export

4.1 Estado global

```
const raw = {
  unidades: null, // CSV fuente principal (unidades sin comunicar)
  inventory: null, // CSV Inventory Report (Guardian/FFC)
  hardware: null, // CSV Listado de Hardware (IMEI GPS + SIM GPS)
  sims: null, // CSV SIMs AT&T (IMEI → ICC)
  pod: null // CSV POD (SIMs secundarias)
};

let exportResult = []; // Array de filas procesadas (output)
let logCount = 0; // Contador de líneas de log
```

4.2 `parseCSV(text)` → `Array<Object>`

Propósito: Parsear texto CSV en array de objetos, tolerante a variantes de formato.

Parámetros:

- `text` (string) — contenido crudo del archivo CSV

Lógica paso a paso:

1. **Strip BOM** — elimina ` ` (Byte Order Mark de archivos Windows)
2. **Detección de separador:**
 - Si la primera línea empieza con `sep=` → usa el carácter indicado (ej: `sep=;`)
 - Sino, cuenta `;` vs `,` → el más frecuente gana. Default `,`
3. **parseRow(line)** — parser de fila con soporte de campos quoted:
 - Mantiene estado `inQ` (dentro de comillas)
 - Maneja comillas dobles escapadas `""` → `"`
 - Acumula `cur` y vacía al encontrar separador fuera de comillas
4. **Sanitización `cv(v)`** — elimina prefijos `=` (artefacto Excel), comillas externas, y trim
5. **Extracción de headers** de la línea `start` (0 o 1 si había `sep=`)
6. **Iteración de filas** — salta líneas vacías o todo-espacios
7. **Construcción de objetos** `{header: value}` por fila

Retorna: Array de objetos `{campo: valor}` donde los keys son los headers del CSV.

4.3 `loadSource(key, inp)`

Propósito: Manejar la carga de un archivo CSV para una fuente específica.

Parámetros:

- `key` (string) — nombre de la fuente: `'unidades' | 'inventory' | 'hardware' | 'sims' | 'pod'`
- `inp` (HTMLInputElement) — input file del DOM

Flujo:

1. Obtiene `File` del input
2. Crea `FileReader`, lee como texto UTF-8
3. En `onload`: llama `parseCSV()`, almacena en `raw[key]`
4. Actualiza UI: clase CSS de la tarjeta (`.fc.ok` o `.fc.err`) y texto de estado
5. Agrega entrada al log

4.4 `normKey(v) → string`

```
const normKey = v => (v || '').replace(/\D/g, '');
```

Propósito: Normalizar IMEIs/ICCs/seriales eliminando todo carácter no numérico. Permite comparar valores que puedan tener guiones, espacios o prefijos de formato distinto.

Ejemplo: `"867-929 064346770"` → `"867929064346770"`

4.5 `buildImeiMap(rows, col) → Map<string, Object>`

Propósito: Construir índice de lookup por IMEI/serial para búsqueda O(1).

Parámetros:

- `rows` — array de objetos CSV
- `col` — nombre de la columna a indexar

Lógica:

- Para cada fila, normaliza el valor de `col` con `normKey`
- Solo indexa valores con 10+ dígitos (descarta vacíos/cortos)
- Retorna `Map<normalizedKey, rowObject>`

4.6 `buildSimMap(simsRows, podRows) → Map<string, string>`

Propósito: Construir índice `IMEI normalizado → ICC/SIM` combinando dos fuentes.

Fuente 1 — SIMs AT&T:

- Columnas: `IMEI` → `ICC`

Fuente 2 — POD (fallback):

- Columnas: `IMEI` → `Subscriber ID #` o `MSISDN`
- Solo inserta si el IMEI no está ya en el mapa (SIMs AT&T tiene prioridad)

4.7 `startProcess()`

Propósito: Orquestrar el procesamiento completo de manera no bloqueante.

Validación: Requiere al menos `raw.unidades` o `raw.inventory`.

Flujo asíncrono con chunking:

1. Deshabilita botón, muestra progress bar
2. `setTimeout(..., 50)` → cede control al navegador para render UI
3. Construye todos los índices: `invMap` (`serial`→row), `invImei` (`imei`→row), `invVeh` (`vehicleName`→row), `hwMap` (`imei GPS`→row), `simMap`
4. Define función `chunk()` que procesa 500 registros por tick:
 - Llama `buildRow()` para cada registro
 - Actualiza progress bar: `(i/total * 85) + 10` (reserva 10% inicio y 5% final)
 - Si quedan registros: `setTimeout(chunk, 0)` (next tick)
 - Si terminó: llama `finalize()`

Por qué chunking: Evita bloquear el thread principal con archivos grandes (>10k filas), manteniendo la UI responsiva.

4.8 `buildRow(r, invMap, invImei, invVeh, hwMap, simMap) → Object`

Propósito: Construir una fila enriquecida del inventario cruzando todas las fuentes.

Parámetros:

- `r` — fila del CSV principal (unidades o inventory)
- `invMap` — índice por serial del Inventory Report
- `invImei` — índice por IMEI Guardian del Inventory Report
- `invVeh` — índice por nombre de vehículo del Inventory Report
- `hwMap` — índice por IMEI GPS del listado de Hardware
- `simMap` — mapa IMEI → SIM

Estrategia de resolución de campos (fallback chain):

```
cuanta:    companyName || account
flota:     fleetName || fleet
vehicle:   vehicleName || vehiclePlate || vehicle || vehicle_registration
serial:    guardianSerial || serial_number
imeiG:     guardianImei || imei
imeiFFC:   ffcImei
imeiGPS:   gpsImei
```

Enriquecimiento desde Inventory Report (si `raw.inventory` existe):

El sistema intenta encontrar la fila correspondiente en el Inventory Report con tres estrategias de lookup en orden de prioridad:

1. Por `serial` (más específico)
2. Por `imeiG` (IMEI Guardian)
3. Por nombre de vehículo (más frágil, fallback)

Si encuentra match → rellena campos vacíos con los del Inventory Report.

Resolución de SIMs:

- `simG` (SIM Guardian): `simMap.get(normKey(imeiG))`
- `simFFC` (SIM FFC Live): `simMap.get(normKey(imeiFFC))`
- `simGPS` (SIM GPS): primero busca en `hwMap` (columnas `simchip` o `phonenumbr`), luego fallback a `simMap`

Clasificación Guardian Generation:

Prefijo del serial	Generación
P1003100...	GEN3
P1002260... O P1001229...	GEN2
P04025...	GEN1
Otro	'' (vacío)

Retorna objeto:

```
{
  cuenta, flota, vehicle, serial,
  imeiG, genGuardian, simG,
  imeiFFC, simFFC, ffcModel,
  imeiGPS, simGPS,
  temporal: '' // campo vacío para completar manualmente
}
```

4.9 finalize()

Ejecutado tras procesar todos los registros:

1. Mueve progress a 100%
2. Calcula estadísticas: total, con serial, con SIM Guardian, con IMEI GPS
3. Actualiza DOM: stats box, botones Exportar/Ver Visor/Limpiar
4. Actualiza badge del nav con conteo total
5. Agrega log de éxito
6. Llama `toast()` con confirmación
7. `setTimeout` 700ms → resetea progress bar a 0%

4.10 doExport()

Propósito: Generar y descargar archivo XLSX con los datos procesados.

Columnas de salida (13 columnas):

#	Header	Campo interno
0	CUENTA	cuenta
1	FLOTA	flota
2	VEHICLEID/PATENTE	vehicle
3	SERIAL GUARDIAN	serial

4	IMEI GUARDIAN	imeiG
5	GEN GUARDIAN	genGuardian
6	SIMCARD GUARDIAN	simG
7	IMEI FFC LIVE	imeiFFC
8	SIMCARD FFC LIVE	simFFC
9	MODELO FFC LIVE	ffcModel
10	IMEI GPS	imeiGPS
11	Sim Card GPS	simGPS
12	¿Temporal o permanente?	temporal

Construcción XLSX manual (sin `sheet_to_json`):

- Crea workbook con `XLSX.utils.book_new()`
- Construye worksheet objeto por celda `{r, c} → {t:'s', v:valor, z:'@'}` (formato texto forzado con `z:'@'` para preservar ceros a la izquierda en IMEIs/ICCs)
- Define `!ref` (rango) y `!cols` (anchos de columna)
- Append sheet como `'Inventario'`
- `XLSX.writeFile()` → descarga directa en el navegador

4.11 `resetExport()`

Restaura toda la UI al estado inicial:

- Limpia `raw`, `exportResult`, `logCount`
- Resetea clases CSS de cada tarjeta de archivo
- Oculta stats, progress, botones secundarios, log
- Limpia valores de inputs file

4.12 Utilidades del módulo Export

`setProgress(pct, msg)`

- Actualiza `width` de `.prog-fill` y textos de porcentaje/mensaje

`addLog(level, msg)`

- `level` — `'ok'` | `'warn'` | `'err'` | `'info'`
- Muestra el panel `.log`
- Crea elemento `.ll.{level}` con timestamp HH:MM:SS
- Auto-scroll al fondo del log body

5. Módulo Visor

5.1 Constantes de columnas `VCOLS`

Array de 13 objetos que define la estructura de la tabla:


```
const VCOLS = [
  { key: 'cuenta',      label: 'CUENTA' },
  { key: 'flota',       label: 'FLOTA' },
  { key: 'vehicle',     label: 'VEHICLEID / PATENTE' },
  { key: 'serial',      label: 'SERIAL GUARDIAN' },
  { key: 'imeiG',       label: 'IMEI GUARDIAN' },
  { key: 'genGuardian', label: 'GEN GUARDIAN' },
  { key: 'simG',        label: 'SIMCARD GUARDIAN' },
  { key: 'imeiFFC',     label: 'IMEI FFC LIVE' },
  { key: 'simFFC',      label: 'SIMCARD FFC LIVE' },
  { key: 'ffcModel',    label: 'MODELO FFC LIVE' },
  { key: 'imeiGPS',     label: 'IMEI GPS' },
  { key: 'simGPS',      label: 'SIM CARD GPS' },
  { key: 'temporal',    label: '¿TEMPORAL O PERMANENTE?' },
];
```

5.2 Definiciones de filtro `FILTER_DEFS`

Array de 10 objetos que configura el sidebar de filtros:

key	label	col (campo)
vehicle	Vehículo / Patente	vehicle
serial	Serial Guardian	serial
imeiG	IMEI Guardian	imeiG
simG	SIMCARD Guardian	simG
imeiFFC	IMEI FFC Live	imeiFFC
simFFC	SIMCARD FFC Live	simFFC
imeiGPS	IMEI GPS	imeiGPS
simGPS	SIM Card GPS	simGPS
cuenta	Cuenta	cuenta
flota	Flota	flota

Cada definición incluye también `icon` (emoji) y `ph` (placeholder con ejemplos).

5.3 Estado del Visor

```
const VF = {};           // { key: Set<string> } – términos activos por filtro
let vAllRows = [];       // todos los registros cargados
let vFiltered = [];      // registros tras aplicar filtros + sort
let vPage = 1;           // página actual
let vPageSize = 50;      // registros por página (opciones: 25/50/100/200)
```

```
let vSortCol = null;    // índice de columna activa en sort (null = sin sort)
let vSortDir = 1;      // 1 = ascendente, -1 = descendente
```

5.4 COL_ALIASES — Mapa de normalización de headers

Diccionario que mapea variantes de nombre de columna (uppercase) a la key interna estandarizada. Soporta múltiples convenciones de naming de diferentes versiones del XLSX:

```
const COL_ALIASES = {
  'CUENTA': 'cuenta',
  'FLOTA': 'flota',
  'VEHICLEID/PATENTE': 'vehicle',
  'VEHICLEID / PATENTE': 'vehicle',
  'VEHICLE': 'vehicle',
  'PATENTE': 'vehicle',
  // ... etc
};
```

Esto hace al Visor tolerante a diferencias de formato entre versiones del archivo exportado.

5.5 buildSidebar()

Genera dinámicamente el HTML del sidebar de filtros a partir de `FILTER_DEFS`. Cada grupo incluye:

- Label con icono
- `<textarea>` con id `fta-{key}` y handler `oninput`
- Hint "Pega varios valores, uno por línea o separados por comas"

Se llama una sola vez al cargar la página (no en cada carga de datos).

5.6 parseTerms(val) → Set<string>

Convierte el valor de un textarea de filtro en un Set de términos normalizados:

- Split por `\n` o `,`
- Trim + lowercase
- Filtra vacíos
- Retorna `Set<string>`

5.7 onVFilterInput(key, val)

Handler de `oninput` de cada textarea. Actualiza `VF[key]` con los términos parseados, resetea a página 1, y llama `applyVFilters()`.

5.8 applyVFilters()

Propósito: Filtrar `vAllRows` según todos los filtros activos + aplicar sort + disparar render.

Lógica de filtrado:

Comportamiento AND entre campos:

Cada campo activo DEBE tener al menos un término que haga match.

Comportamiento OR dentro de un campo:

Si el campo "vehicle" tiene términos ["PSVR54", "PSVR56"],
la fila pasa si vehicle contiene "PSVR54" OR "PSVR56".

Match type: substring case-insensitive (includes).

Sort (si `vSortCol !== null`):

- Ordena `vFiltered` por `VCOLS[vSortCol].key`
- Comparación string lowercase
- Dirección: `vSortDir` (+1 asc, -1 desc)

Post-filtrado: Llama `renderVTable()` y `updateVInfo()`.

5.9 renderVTable()

Propósito: Renderizar la tabla paginada con highlight de términos buscados.

Flujo:

1. Si `vFiltered` vacío → renderiza empty state con mensaje contextual
2. Calcula slice de página: `start = (vPage-1) * vPageSize`, `end = min(start+vPageSize, total)`
3. Construye mapa `termsByCol` — para cada columna, array de términos activos en ese filtro
4. `hlCell(val, colKey)` — renderiza celda con highlight:
 - Celda vacía → `<span class="empty">—</span>`
 - Columna `genGuardian` → span con clase `.gen1/.gen2/.gen3` (colores amber/cyan/green)
 - Busca primer término activo en el valor con `indexOf case-insensitive`
 - Si match: envuelve en `<mark class="hl">` (fondo cyan tenue)
 - Si no match: `esc(v)` (solo escape HTML)
5. `thSort(label, ci)` — renderiza `<th>` con clase `asc/desc` según estado de sort
6. Construye HTML completo: header con meta-info y botón exportar, tabla, pager

5.10 buildVPager(total) → string

Genera HTML del paginador con lógica de páginas visibles:

- ≤7 páginas → muestra todas
- 7 páginas → muestra `[1, ..., vPage-2..vPage+2, ..., Last]` con `...` como separadores
- Botones « (primera), < (anterior), números, > (siguiente), » (última)
- Página actual con clase `.pg.on`
- Botones de extremo deshabilitados cuando en primera/última página

5.11 Carga de archivos en el Visor

Drag & Drop:

- `vDragOver(e)` → `e.preventDefault()` + clase `.drag` al drop zone
- `vDragLeave(e)` → quita clase `.drag`
- `vDrop(e)` → previene default, extrae `e.dataTransfer.files[0]`, llama `vReadFile()`

File input:

- `vFileChange(inp)` → extrae primer archivo, llama `vReadFile()`

vReadFile(file) :

- `FileReader.readAsArrayBuffer()` (necesario para XLSX)
- En `onload` : `XLSX.read(data, {type:'array', raw:false})`
- Toma primer sheet, convierte a array 2D con `sheet_to_json({header:1})`
- Pasa a `vParseRows()`

`vParseRows(raw, fname) :`

1. Detecta fila de headers: busca en primeras 5 filas la que tenga ≥ 3 columnas reconocidas por `COL_ALIASES`
2. Mapea columnas a keys internas
3. Construye `vAllRows` (objetos con las 13 keys de `VCOLS`)
4. Salta filas completamente vacías
5. Llama `clearFilters()` → resetea todos los filtros y textareas
6. Muestra layout Visor, oculta drop zone
7. Actualiza badge del nav
8. Llama `applyVFilters()` → render inicial

`loadVisorFromMemory(rows) :`

- Alternativa a cargar desde archivo: recibe los datos procesados de la pestaña Export directamente
- Clona el array (`{...r}`) para evitar mutación cruzada
- Mismo flujo de estado que `vParseRows`

5.12 `exportVFiltered()`

Exporta a XLSX solo los registros actualmente visibles en el filtro (no toda la data):

- Mismas 13 columnas que `doExport()`
- Nombre de archivo: `inventario_filtrado_{N}_{fecha}.xlsx` si hay filtros activos, `inventario_completo_{fecha}.xlsx` si no
- Sheet name: `'Filtrado'`

5.13 Utilidades compartidas

`esc(s) → string`

```
const esc = s => String(s || '').replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
```

Escapa caracteres especiales HTML para prevenir XSS en renderizado dinámico.

`toast(msg, type)`

- `type` — `'ok' | 'err' | ''`
- Agrega clase `.show` + tipo al elemento `#toast`
- Limpia timer anterior (`toastTimer`) para evitar desapariciones prematuras
- Desaparece tras 3500ms

6. Flujo de Datos Completo

Flujo Export

```
Usuario carga CSV(s)
└─ loadSource(key, inp)
```

```
└─ FileReader.readAsText()
└─ parseCSV(text) → raw[key]
```

Usuario hace clic "Procesar y cruzar datos"

```
└─ startProcess()
  └─ buildImeiMap(inventory, 'serial_number') → invMap
  └─ buildImeiMap(inventory, 'imei') → invImei
  └─ Map vehículo→row → invVeh
  └─ buildImeiMap(hardware, 'imei') → hwMap
  └─ buildSimMap(sims, pod) → simMap
  └─ chunk() × N veces (500 registros/tick)
    └─ buildRow(r, invMap, invImei, invVeh, hwMap, simMap)
      └─ exportResult.push(row)
    └─ finalize() al terminar
```

Usuario hace clic "Exportar XLSX"

```
└─ doExport()
  └─ XLSX.writeFile() → descarga navegador
```

Usuario hace clic "Ver en Visor"

```
└─ goToVisor()
  └─ loadVisorFromMemory(exportResult)
  └─ switchTab('visor')
```

Flujo Visor

Usuario arrastra/selecciona XLSX

```
└─ vReadFile(file)
  └─ FileReader.readAsArrayBuffer()
  └─ XLSX.read() → sheet 2D array
  └─ vParseRows()
    └─ detecta fila de headers (COL_ALIASES)
    └─ construye vAllRows
    └─ applyVFilters() → renderVTable()
```

Usuario escribe en textarea de filtro

```
└─ onVFilterInput(key, val)
  └─ parseTerms(val) → VF[key]
  └─ applyVFilters()
    └─ filtra vAllRows (AND entre campos, OR dentro)
    └─ sort si vSortCol activo
    └─ renderVTable() + updateVInfo()
```

Usuario hace clic en header de columna

```
└─ vSort(ci)
  └─ toggle dirección si misma col, else reset a asc
  └─ applyVFilters() → re-render con nuevo sort
```

Usuario cambia de página

```
└─ vGoPage(p) → vPage = clamp(p, 1, maxPage) → renderVTable()
```

```
Usuario exporta vista filtrada
└─ exportVFiltered() → XLSX.writeFile()
```

7. Fuentes de Datos — Estructura Esperada de CSVs

7.1 Unidades sin comunicar (fuente principal)

Columnas utilizadas (con fallbacks):

Campo interno	Columnas buscadas en CSV
cuenta	companyName, account
flota	fleetName, fleet
vehicle	vehicleName, vehiclePlate, vehicle, vehicle_registration
serial	guardianSerial, serial_number
imeiG	guardianImei, imei
imeiFFC	ffcImei
imeiGPS	gpsImei
ffcModel	ffcModel, ffc_model

7.2 Inventory Report (Guardian / FFC)

Columnas utilizadas para indexado:

- `serial_number` — clave primaria de búsqueda
- `imei` — clave secundaria de búsqueda
- `vehicle` / `vehicleName` — clave terciaria (fallback)
- `serial_number` , `guardianSerial` , `imei` , `guardianImei` , `ffcImei` , `gpsImei` — datos a enriquecer

7.3 Listado de Hardware (IMEI GPS)

Columnas utilizadas:

- `imei` — clave de indexado
- `simchip` o `phonenumber` — SIM asociada al GPS

7.4 SIMs AT&T

Columnas utilizadas:

- `IMEI` — clave de indexado
- `ICC` — número de SIM card

7.5 POD (SIMs secundarias)

Columnas utilizadas:

- `IMEI` — clave de indexado

- `Subscriber ID #` o `MSISDN` — número de SIM (fallback si no está en AT&T)
-

8. Consideraciones Técnicas y Limitaciones

8.1 Procesamiento client-side

Todo el procesamiento ocurre en el navegador del usuario. Ventajas:

- Sin servidor, sin backend, sin red (funciona offline)
- Los archivos CSV/XLSX nunca salen del dispositivo

Limitaciones:

- El navegador puede quedarse sin memoria con archivos muy grandes (>100k filas)
- No hay persistencia: al refrescar la página se pierde todo

8.2 Tolerancia a formatos

El sistema fue diseñado para ser resiliente a variaciones de formato:

- BOM () en archivos Windows
- Separadores , o ; (auto-detección)
- Prefijos = de Excel en campos numéricos
- Campos entre comillas con comas internas
- Headers con distintas capitalizaciones (COL_ALIASES normaliza a uppercase)
- IMEI con o sin guiones (normKey elimina no-numéricos)

8.3 Normalización de IMEIs

Todos los lookups por IMEI/ICC/serial numérico pasan por `normKey()` . El criterio de validez mínima es 10 dígitos (descarta valores nulos o truncados).

8.4 Prioridad de fuentes de datos

Para un campo dado, el orden de prioridad es:

1. CSV de Unidades sin comunicar (row de la fuente principal)
2. Inventory Report (enriquecimiento por serial > IMEI > vehículo)

Para SIM cards:

1. SIMs AT&T (`sims`)
2. POD (`pod`) — solo si no está en AT&T

Para SIM del GPS específicamente:

1. Listado de Hardware (columnas `simchip` / `phonenumber`)
2. simMap general (AT&T o POD)

8.5 Seguridad

- `esc()` previene XSS en el render de la tabla
 - No hay evaluación de código externo
 - Sin peticiones de red (excepto fuentes Google Fonts en carga inicial)
-

9. Propuesta de Migración a React

9.1 Stack tecnológico propuesto

```
React 18 + TypeScript + Vite
├─ Tailwind CSS (estilos utilitarios)
├─ shadcn/ui (componentes UI)
├─ xlsx (misma librería, importada como módulo)
└─ React Router v6 (opcional, para rutas /export y /visor)
```

No hay "Java" en la ecuación. La respuesta a tu pregunta es:

React + TypeScript + CSS (Tailwind) = todo el stack frontend.

El HTML sirve como plantilla visual, CSS como referencia de diseño, y el JavaScript se convierte en componentes React, hooks, y utilidades TypeScript.

9.2 Estructura de carpetas propuesta

```
fleet-inventory/
├─ index.html                # Shell mínimo (solo <div id="root">)
├─ vite.config.ts
├─ tsconfig.json
├─ tailwind.config.ts
├─ components.json           # Config de shadcn/ui
├─ package.json
├─
└─ src/
    ├─ main.tsx              # Entry point: ReactDOM.render(<App/>)
    ├─ App.tsx               # Layout raíz + tab state
    │
    ├─ types/
    │   └─ index.ts          # Interfaces: InventoryRow, RawSources, FilterState
    │
    ├─ lib/
    │   ├─ csvParser.ts      # parseCSV(), loadSource()
    │   ├─ dataProcessor.ts  # buildRow(), buildImeiMap(), buildSimMap()
    │   ├─ exportXLSX.ts     # doExport(), exportVFiltered()
    │   ├─ constants.ts     # VCOLS, FILTER_DEFS, COL_ALIASES, HDRS
    │   └─ utils.ts          # normKey(), esc(), toast helpers
    │
    ├─ hooks/
    │   ├─ useCSVLoader.ts   # Estado de raw sources + loadSource
    │   ├─ useDataProcessor.ts # startProcess(), exportResult, progress
    │   ├─ useVisorFilters.ts # VF state, applyVFilters, sort, pagination
    │   └─ useToast.ts       # toast state + timer
    │
    ├─ components/
    │   └─ layout/
    │       ├─ Nav.tsx       # Barra de navegación sticky
    │       └─ NavBadge.tsx  # Chip contador de registros
    │
    │   └─ export/
    │       └─ ExportPage.tsx # Página completa Export
```



```

├─── FileCard.tsx           # Tarjeta individual de archivo CSV
├─── FileGrid.tsx          # Grid de 5 FileCards
├─── StatsBox.tsx          # Cuatro métricas de resultado
├─── ProgressBar.tsx       # Barra de progreso animada
├─── ActionButtons.tsx     # Procesar / Exportar / Ver Visor / Limpiar
├─── LogBox.tsx            # Panel de log con timestamps
├─── visor/
│   ├── VisorPage.tsx      # Página completa Visor
│   ├── VisorDropZone.tsx  # Drop zone drag & drop
│   ├── FilterSidebar.tsx  # Sidebar con todos los grupos de filtro
│   ├── FilterGroup.tsx    # Textarea individual de filtro + chips
│   ├── DataTable.tsx      # Tabla con headers sortables
│   ├── TableRow.tsx       # Fila con highlight de términos
│   └── Pager.tsx          # Paginador con ellipsis
├─── ui/                   # Componentes shadcn/ui (auto-generados)
│   ├── button.tsx
│   ├── badge.tsx
│   ├── card.tsx
│   ├── select.tsx
│   ├── textarea.tsx
│   └── toast.tsx (Sonner)
├─── styles/
│   ├── globals.css        # Variables CSS + Tailwind base
│   └── theme.ts           # Tokens del design system (colores, etc.)

```

9.3 Separación de responsabilidades

Qué es hoy (HTML monolítico)	Qué será en React
Variables CSS en :root	styles/globals.css + tailwind.config.ts (tokens)
Estilos inline en <style>	Clases Tailwind + componentes shadcn/ui
raw, exportResult, logCount	useCSVLoader.ts + useDataProcessor.ts (custom hooks)
parseCSV(), normKey()	lib/csvParser.ts, lib/utlis.ts (funciones puras)
buildRow(), buildImeiMap()	lib/dataProcessor.ts
VCOLS, FILTER_DEFS, COL_ALIASES	lib/constants.ts
VF, vAllRows, vFiltered, etc.	useVisorFilters.ts
renderVTable() (innerHTML)	DataTable.tsx (JSX declarativo)
toast() con timer global	useToast.ts + Sonner (librería de toasts)
switchTab() con display:block/none	Estado en App.tsx o React Router

10. Glosario

Término	Definición
Guardian	Dispositivo telemático principal instalado en el vehículo
FFC Live	Módulo adicional de comunicación (Free Fleet Connect)
GPS	Dispositivo GPS independiente
IMEI	International Mobile Equipment Identity — identificador único de dispositivo
ICC / SIM	Integrated Circuit Card — número de la tarjeta SIM
Serial / S/N	Número de serie del dispositivo Guardian
GEN1/2/3	Generación del hardware Guardian (determinada por prefijo del serial)
POD	Fuente secundaria de SIMs (posiblemente "Proof of Delivery" o sistema interno)
Patente	Placa/matrícula del vehículo (terminología latam)